

Chapter 7

Oops concept

What is packages and advantage?

package in Java is used to group related classes, interfaces, enums, and sub-packages. Packages help organize code, avoid naming conflicts, control access, and make large applications easier to maintain.

A package can contain classes, interfaces, enums, and sub-packages.

The package statement defines the package of a source file.

They make code easier to maintain and reuse.

Types of Java Packages

Java packages are mainly divided into two types:

1. Built-in Packages:

```
import java.util.Random;    // built-in package
```

```
public class GFG{
```

```
    public static void main(String[] args) {
```

```
        // using Random class
```

```
        Random rand = new Random();
```

```
        // generates a number between 0–99
```

```
        int number = rand.nextInt(100);
```

```
        System.out.println("Random number: "
+ number);
```

```
    }
```

```
}
```

2. User-defined Packages:

```
package com.myapp;

public class Helper {
    public static void show() {
        System.out.println("Hello      from
Helper!");
    }
}
```

```
import java.util.Vector;
```

```
public class Geeks {
```

```
    public Geeks() {
```

```
        // java.util.Vector is imported, We are
        able to access it directly in our code.
```

```
        Vector v = new Vector();
```



```
        java.util.ArrayList l = new
java.util.ArrayList();
        l.add(3);
        l.add(5);
        l.add(7);

        System.out.println(l);
    }

    public static void main(String[] args) {

        new Geeks();
    }
}
```

Static Import:

In Java, static import concept is introduced in 1.5 version. With the help of static import, we can access the static members of a class directly without class name or any object. For Example: we always use `sqrt()` method of `Math` class by using `Math` class i.e. `Math.sqrt()`, but by using static import we can access `sqrt()` method directly.

According to SUN microSystem, it will improve the code readability and enhance coding. But according to the programming experts, it will lead to confusion and not good for programming. If there is no specific requirement then we should not go for static import.

Advantage of static import:

If user wants to access any static member of class then less coding is required.

Disadvantage of static import:

Static import makes the program unreadable and unmaintainable if you are reusing this feature, especially in large codebases or projects with multiple developers.

```
// Java Program to illustrate
// calling of predefined methods
// without static import
class Geeks {
    public static void main(String[] args)
    {
        System.out.println(Math.sqrt(4));
        System.out.println(Math.pow(2, 2));
        System.out.println(Math.abs(6.3));
    }
}
```


ACCESS MODIFIERS :

Access modifiers in Java are used to control the visibility and accessibility of classes, methods, and variables. They help enforce encapsulation by restricting access to different parts of a program. Java provides four types of access modifiers to define scope and protection levels.

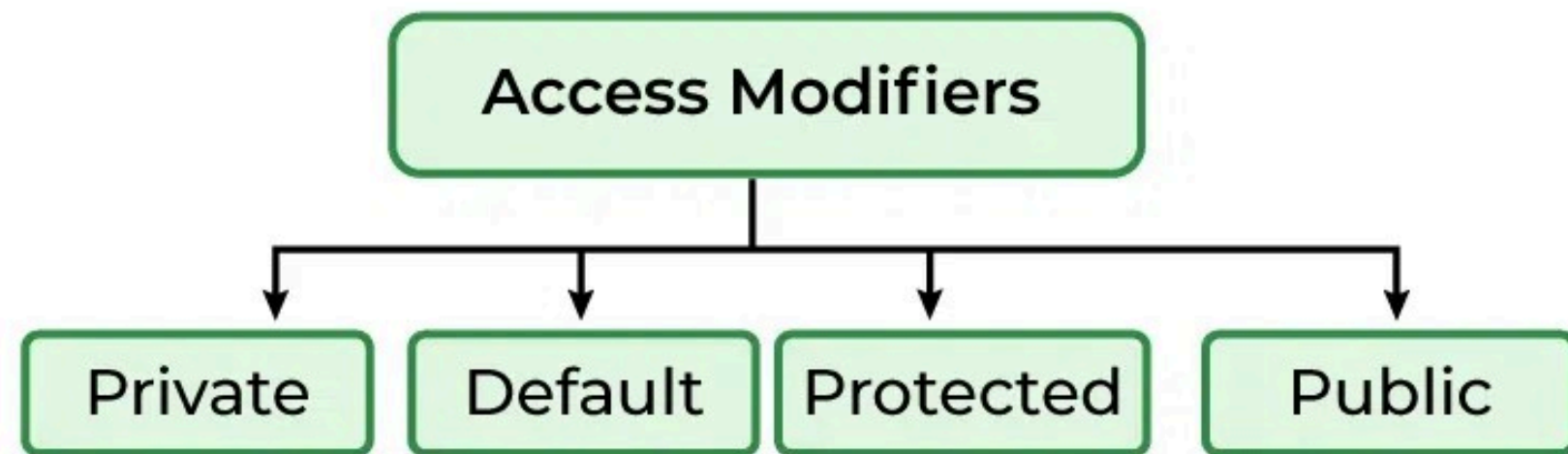
Public modifier: Accessible from anywhere in the program

Protected modifier: Accessible within the same package and by subclasses

Private modifier: Accessible only within the same class

Default modifier: Accessible only within the same package

Access Modifiers in Java



```
class Person {
```

```
    // private variable
```

```
    private String name;
```

```
    public void setName(String name) {
```

```
        this.name = name; // accessible  
        within class  
    }
```

```
    public String getName() { return name; }  
}
```



```
public class Geeks {  
    public static void main(String[] args)  
    {  
  
        Person p = new Person();  
        p.setName("Alice");  
  
        // System.out.println(p.name); // Error:  
'name'  
  
        // has private access  
        System.out.println(p.getName());  
    }  
}
```

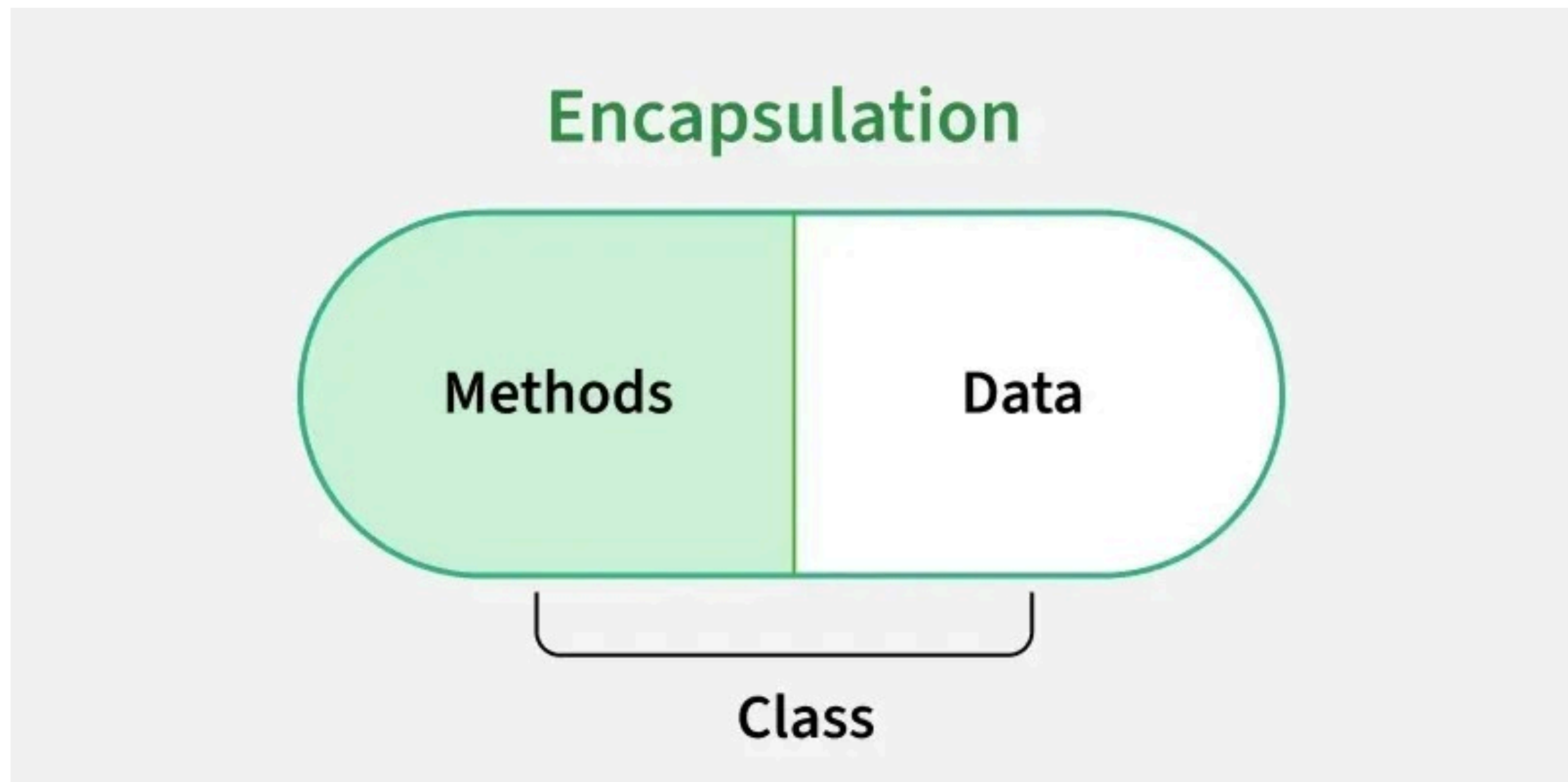
ENCAPSULATION :

Encapsulation in Java is an object-oriented programming concept that bundles data and the methods that operate on that data into a single unit, such as a class. It also helps control access to the object's internal state by restricting direct access and providing controlled ways to read or modify it.

Usually uses private fields to restrict direct access.

Provides controlled access through methods such as getters and setters when required.

Allows validation and other rules to be applied before changing data.



Abstraction :

Abstraction in Java is the process of hiding internal implementation details and showing only essential functionality to the user. It focuses on what an object does rather than how it does it.

It hides the complex details and shows only essential features.

Abstract classes may have methods without

implementation and must be implemented by subclasses.

By abstracting functionality, changes in the implementation do not affect the code that depends on the abstraction.

Polymorphism :

Polymorphism is one of the fundamental concepts of Object-Oriented Programming (OOP) that allows a single entity to take multiple forms. It enables the same method or interface to behave differently depending on the object involved. Polymorphism improves code flexibility, reusability, and maintainability.

Polymorphism means "many forms".

It allows the same method to exhibit different behaviors.

It is achieved through method overloading and method overriding.

Example: Consider a person who plays different roles in life, like a father, a husband, and an employee. Each of these roles defines different behaviors of the person depending on the object calling it.

Program :

```
class Person {  
  
    // Method that displays the  
    // role of a person  
    void role() { System.out.println("I am a  
person."); }  
}  
  
// Derived class Father that  
// overrides the role method  
class Father extends Person {  
  
    // Overridden method to show
```

```
// the role of a father
@Override void role()
{
    System.out.println("I am a father.");
}
}

public class Main {
    public static void main(String[] args)
    {

        // Creating a reference of type Person
        // but initializing it with Father class
        object
        Person p = new Father();

        // Calling the role method. It calls the
        // overridden version in Father class
        p.role();
    }
}
```